

Pregunta 5) inciso A

```
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# 1. Definir el ticker y rango de fechas
ticker_symbol = "^GSPC"# Ejemplo: S&P 500 Index. Puedes cambiar a
'EURUSD=X' para Euro/USD o 'AMZN' para Amazon.
start_date = "2000-01-01"
end_date = "2024-01-01"

# 2. Descargar los datos diarios
data = yf.download(ticker_symbol, start=start_date, end=end_date,
progress=False)["Close"]

# 3. Resamplear a frecuencia mensual (primer valor del mes) y eliminar NAs
monthly_series = data.resample("MS").first().dropna()
monthly_series.name = f"{ticker_symbol} Cierre Mensual"

# 4. Mostrar las primeras filas de la serie mensual
print(f"Serie financiera mensual de {ticker_symbol}:")
display(monthly_series.head())

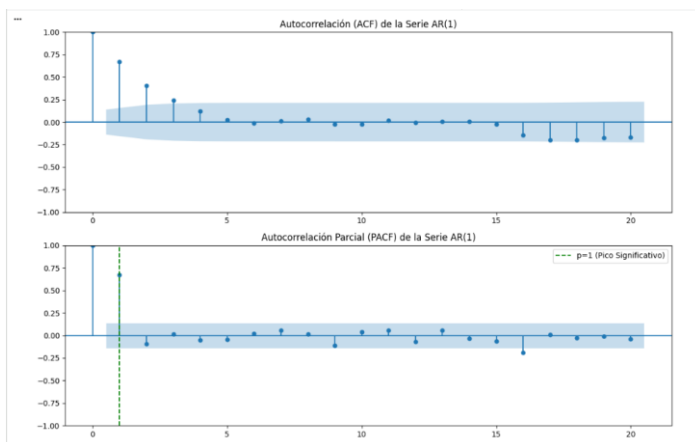
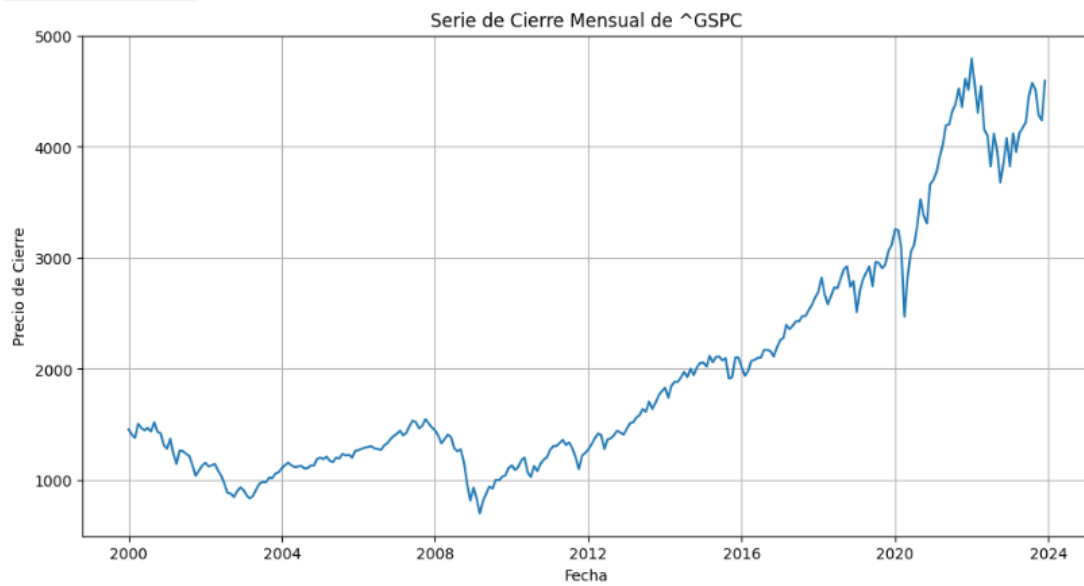
# 5. Visualización rápida de la serie
plt.figure(figsize=(12, 6))
plt.plot(monthly_series)
plt.title(f'Serie de Cierre Mensual de {ticker_symbol}')
plt.xlabel('Fecha')
```

```
plt.ylabel('Precio de Cierre')
```

```
plt.grid(True)
```

```
plt.show()
```

Ticker	^GSPC
Date	
2000-01-01	1455.219971
2000-02-01	1409.280029
2000-03-01	1379.189941
2000-04-01	1505.969971
2000-05-01	1468.250000



Pregunta 6) Inciso A

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# 1. Simulación de una serie mensual de ventas (ejemplo para ilustrar)
# Esta simulación incluye tendencia y estacionalidad.
n_months = 12 * 10 # 10 años de datos mensuales
np.random.seed(45) # Para reproducibilidad

# Generar índice de fechas mensuales
dates_monthly = pd.date_range(start='2010-01-01', periods=n_months,
                               freq='MS')

# Tendencia lineal creciente
trend = np.linspace(500, 1500, n_months)

# Estacionalidad anual (ciclo de 12 meses)
# Usaremos una onda sinusoidal que se repite anualmente
seasonal_amplitude = 100
seasonal = seasonal_amplitude * np.sin(np.linspace(0, 2 * np.pi *
(n_months / 12), n_months) + np.pi/2) # +pi/2 para empezar en pico

# Ruido aleatorio
noise = np.random.normal(0, 30, n_months)

# Combinar componentes para la serie de ventas
```

```
sales_series_simulated = pd.Series(trend + seasonal + noise,  
index=dates_monthly, name='Ventas Mensuales Simuladas')
```

```
# 2. Visualización de la serie simulada y sus ACF/PACF
```

```
fig, axes = plt.subplots(3, 1, figsize=(14, 12))
```

```
sales_series_simulated.plot(ax=axes[0], title='Serie de Ventas Mensuales  
Simulada (con Tendencia y Estacionalidad)', color='blue')
```

```
axes[0].set_xlabel('Fecha')
```

```
axes[0].set_ylabel('Ventas')
```

```
axes[0].grid(True)
```

```
plot_acf(sales_series_simulated, lags=36, ax=axes[1], title='ACF de la  
Serie de Ventas Simulada')
```

```
axes[1].set_xlabel('Lag')
```

```
axes[1].set_ylabel('Autocorrelación')
```

```
plot_pacf(sales_series_simulated, lags=36, ax=axes[2], title='PACF de la  
Serie de Ventas Simulada', method='ywm')
```

```
axes[2].set_xlabel('Lag')
```

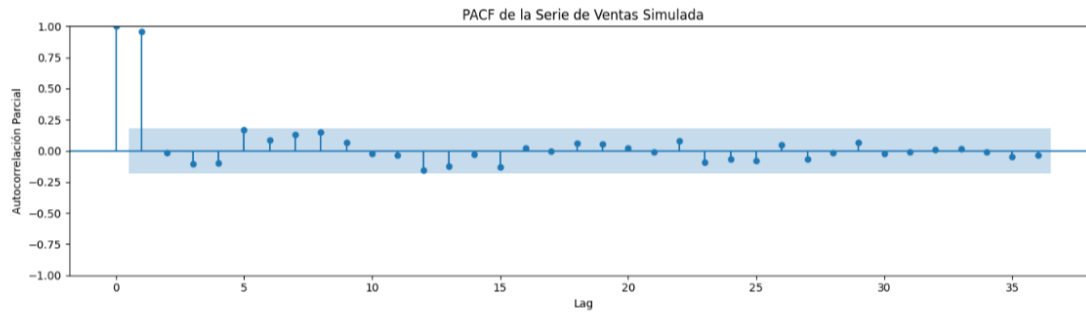
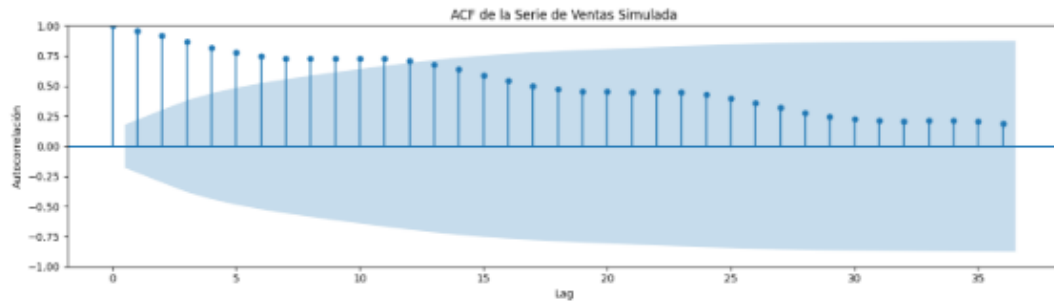
```
axes[2].set_ylabel('Autocorrelación Parcial')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
display(sales_series_simulated.head())
```

—



Ventas Mensuales Simuladas	
2010-01-01	600.791243
2010-02-01	602.594716
2010-03-01	554.188330
2010-04-01	517.761098
2010-05-01	443.948087